1.2.31

Sai Sreevallabh - EE25BTECH11031

Question

The smallest degree of the polynomial that interpolates the given data is

x	-2	-1	0	1	2	3
$f(x)$	-58	-21	-12	-13	-6	27

Finite Differences Method

The method of Finite Differences is a method used to determine the degree of a polynomial when given the value of the polynomial at equally spaced inputs, as given in the question.

It involves calculating the difference between consecutive y-values to create a new sequence called "first differences." This process is repeated on the newly formed sequence to find "second differences," then "third differences," and so on.

The degree of teh polynomial is n if all the nth differences are equal.

Why this Method Works

These finite differences are the finite, discrete analogue of derivatives.

We know that an nth degree polynomial has zero nth derivative.

Similarly, an n degree polynomial will have the nth differences as same.

By consecutively calculating the differences until a constant level is reached, we can determine the degree of the polynomial.

Theoretical Solution

x	$f(x)$	1st Diff (Δf)	2nd Diff ($\Delta^2 f$)	3rd Diff ($\Delta^3 f$)
-2	-58			
		-21 - (-58) = **37**		
-1	-21		9 - 37 = **-28**	
		-12 - (-21) = **9**		-10 - (-28) = **18**
0	-12		-1 - 9 = **-10**	
		-13 - (-12) = **-1**		8 - (-10) = **18**
1	-13		7 - (-1) = **8**	
		-6 - (-13) = **7**		26 - 8 = **18**
2	-6		33 - 7 = **26**	
		27 - (-6) = **33**		
3	27			

Lagrange's Method of Interpolation:

Interpolation of a graph is the mathematical method of estimating unknown, intermediate data points between a discrete set of known data points.

The Lagrange method for interpolation is widely used. This gives the expression for the lowest degree polynomial which passes through all the given data points.

This is implemented in the scipy.interpolate library.

Frame Title

This method used to find the unique polynomial of lowest degree that fits a set of data points, regardless of whether the x-values are equally spaced.

This is achieved by building specific basis polynomials, $L_i(x)$, which act as switches, going to 1 at their corresponding data point x_i, and 0 at all other known x-values in the set.

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^{n} \frac{x - x_j}{x_i - x_j}$$

Lagrange's Method For Interpolation

The required polynomial can be given by

$$P(x) = \sum_{i=0}^{n} y_i L_i(x)$$

This method is tedious by calculating with hand, but gives the correct curve.

However, for very large number of data points, if a high degree polynomial must be plotted, the plot exhibits violent oscillations near the edges of the interval. This instability is known as Runge's Phenomenon.

Cubic Splines

- To counter that problem, Cubic Splines are used.
- Instead of a single global equation, a spline constructs a piecewise function made of multiple low-degree polynomials (degree 3 for cubic spline).
- The segments are stitched together at the data points keeping in mind certain rules, to ensure that the overall curve has continuous slope and curvatures.
- This is the method that is used for the interpolation of a graph accurately using SciPy.

In the simulations provided, the number of data points is not high and hence, both Cubic Splines and Lagrange's method give the same graph.

Code for Finite Differences

```
import numpy as np

y = np.array([-58, -21, -12, -13, -6, 27])

#loop for calculating finite differences
for n in range(1, len(y)):

    current_diff = np.diff(y, n=n)
    print(f"{n} - Difference: {current_diff}")

    if np.all(current_diff == current_diff[0]):
        print(f"The smallest degree of the polynomial is: {n}")
        break
```

The above code uses the inbuilt finite differences calculator present in numpy: np.diff.
The code outputs the finite differences sequentially and the smallest degree of the polynomial.

Simulation Code: Interpolation

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.interpolate import lagrange, CubicSpline

x = np.array([-2, -1, 0, 1, 2, 3])
y = np.array([-58, -21, -12, -13, -6, 27])

#Finding the polynomial using lagrange's method
poly_lagrange = lagrange(x, y)

final_coeffs = [c for c in poly_lagrange.coeffs if abs(c) > 1e
    -10]
clean_poly = np.poly1d(final_coeffs)
true_degree = clean_poly.order

poly_spline = CubicSpline(x,y)
```

Simulation Code: Interpolation

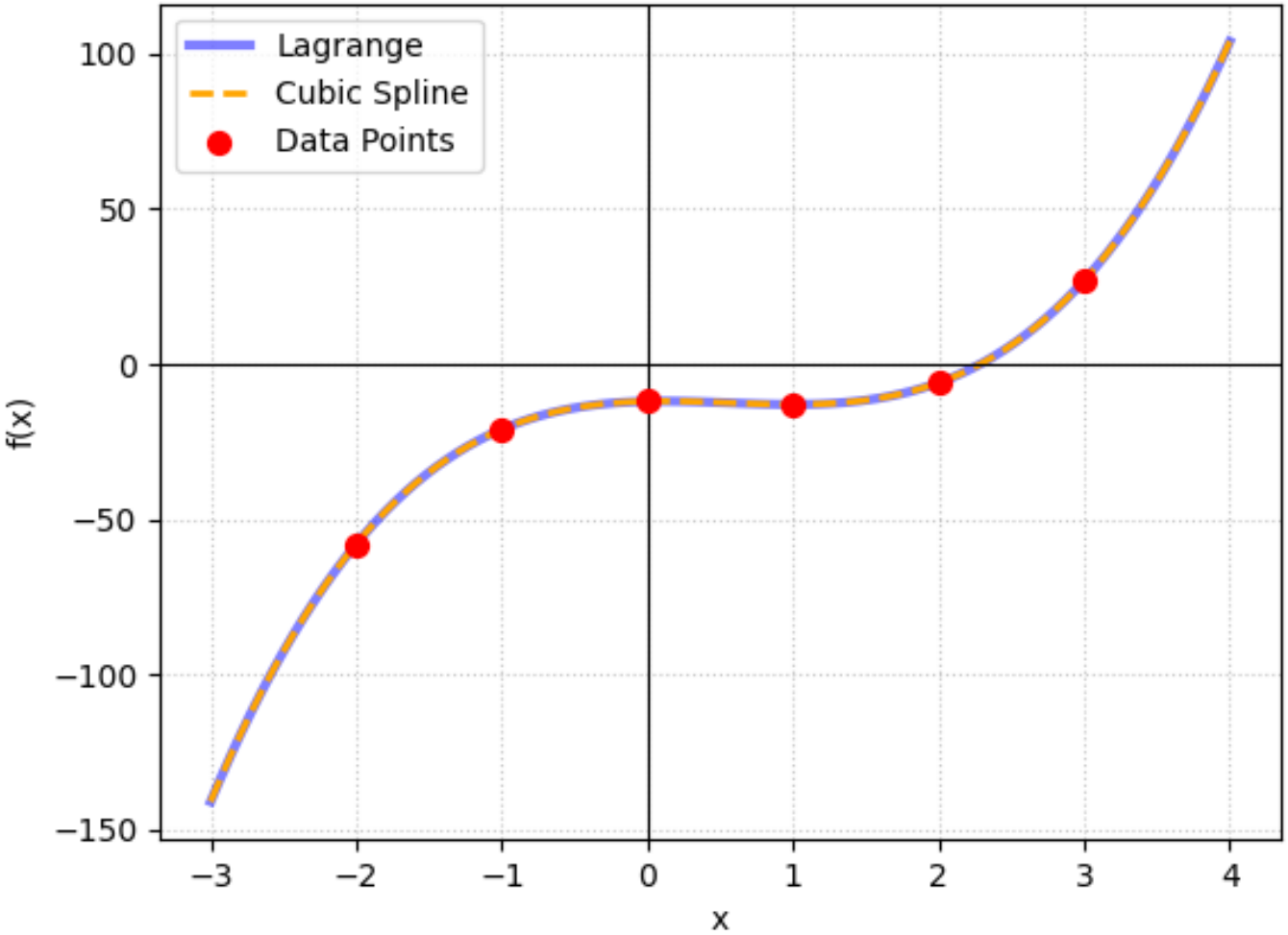
```
x_smooth = np.linspace(-3, 4, 100)

plt.plot(x_smooth, clean_poly(x_smooth), color='blue', linewidth
    =3, alpha=0.5,
         label=f'Lagrange')

plt.plot(x_smooth, poly_spline(x_smooth), color='orange',
    linestyle='--', linewidth=2,
         label='Cubic Spline')
plt.scatter(x, y, color='red', s=50, zorder=5, label='Data Points
    ')
```

Simulation Code: Interpolation

```
plt.xlabel('x')
plt.ylabel('f(x)')
plt.axhline(0, color='black', linewidth=0.8) # Draws the x-axis
plt.axvline(0, color='black', linewidth=0.8) # Draws the y-axis
plt.grid(True, linestyle=':', alpha=0.7)
plt.legend()

# Display the graph
plt.savefig("../Figs/plot.png")
plt.show()
```

Simulation Plot